Chapitre 0 — Avant tout | Partie A — Écrire des lettres | leçon 0.8.5 sur 153 | difficulté 0 / 10

0.8.5. #include <diviser> — a / b

#include <diviser> : a / b demande une routine, sauf par 1, 2, 4, 8, 16… écrits en clair.

1. Ce qu'il faut comprendre

Ce que c'est : le processeur **ne sait pas diviser** non plus. Diviser par 2, 4, 8, 16… écrit en clair, c'est décaler les bits : le compilateur le fait seul. Par **tout autre nombre**, ou par une valeur calculée, il faut une routine.

La ligne à écrire : `#include <diviser>`**.** Le résultat est **entier** : 100 / 7 vaut 14, le reste se perd.

Ce qu'elle coûte : environ **30 octets**.

Essaie : `a / 4` sans la ligne : ça compile.

> **On retrouve diviser dans 5 leçons**
>
> 0.9, 0.9.1, 0.9.2, 10, 24

📖 Les mots de cette leçon — si tu débutes

- **accolades { }** — Elles entourent un bloc : les lignes qui vont ensemble (le corps d'une fonction, d'une boucle, d'un « if »). Chaque « { » a son « } ».
- **affectation (=)** — « x = 5; » : la boîte x prend la valeur 5. Ce n'est pas une égalité : c'est un rangement.
- **argument** — Une valeur donnée à une fonction quand on l'appelle, entre ses parenthèses : dans texte(5, 8, "SALUT"), 5, 8 et "SALUT" sont les arguments.
- **bloc** — Un groupe de lignes entre accolades { }, qui s'exécutent ensemble.
- **booléen (vrai / faux)** — Le résultat d'une condition : vrai (1) ou faux (0). « true » veut dire vrai, « false » faux.
- **boucle** — Des instructions qui se répètent : « while » (tant que…) ou « for » (pour un compteur qui va de… à…).
- **boucle du jeu** — « while (true) { image(); … } » : ce qui se répète 60 fois par seconde, sans fin, pendant tout le jeu.
- **commentaire (//)** — Tout ce qui suit « // » jusqu'au bout de la ligne : une phrase pour l'humain, que la console ignore.
- **condition** — Une question dont la réponse est vrai ou faux : « score > 10 », « bouton(A) ». On la teste avec « if » ou « while ».
- **coordonnées** — Deux nombres qui disent où : la colonne (vers la droite) puis la ligne (vers le bas), à partir de (0, 0) en haut à gauche.
- **image()** — Attend l'image suivante. La console en montre 60 par seconde : c'est ce qui règle la vitesse d'un jeu.
- **#include** — En haut du programme : « #include <texte> » demande la fonction texte() à la console ; « #include "fichier.cpp" » verse un de tes fichiers à cet endroit.
- **instruction** — Un ordre donné à la console, terminé par un point-virgule : « texte(1, 1, "OK"); ».
- **main** — « int main() { … } » : là où le programme commence. La console exécute d'abord la première ligne de main.
- **octet** — Un nombre de 0 à 255 : la plus petite boîte de la mémoire. Une cartouche en a 32 768.
- **opérateurs de calcul** — + plus, - moins, * fois, / divisé par (sans virgule : 7 / 2 donne 3), % le reste de la division (7 % 2 donne 1).
- **point-virgule (;)** — Il termine une instruction, comme le point termine une phrase.
- **uint8_t** — Le genre de nombre de cette console : un entier de 0 à 255 (un octet). Au-delà, il repart à 0.
- **variable** — Une boîte avec un nom, dans la mémoire, qui retient un nombre qu'on peut lire et changer.
- **while (tant que)** — Répète un bloc tant qu'une condition est vraie.

🔍 Le programme, ligne par ligne — chaque ligne dite en français

`// ---- #include <diviser> : a / b ----`	Un commentaire : une phrase pour celui qui lit (« ---- #include <diviser> : a / b ---- »). La console l'ignore.
`#include <diviser>  // la routine qui divise`	Demande à la console « diviser » (ce que c'est : a / b, sauf par 1, 2, 4, 8, 16… écrits en clair). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « la routine qui divise ».)
`#include <nombre>`	Demande à la console « nombre » (ce que c'est : écrit un nombre en chiffres). Sans cette ligne, le compilateur refuserait de l'employer.

`uint8_t a = 100;`	Une variable : une boîte appelée a, qui retient un nombre de 0 à 255, et qui commence à 100.
`uint8_t b = 7;`	Une variable : une boîte appelée b, qui retient un nombre de 0 à 255, et qui commence à 7.
`uint8_t quotient = 0;`	Une variable : une boîte appelée quotient, qui retient un nombre de 0 à 255, et qui commence à 0.
`int main() {`	Le début du programme : la console commence ici, par la première ligne qui suit.
`quotient = a / b;  // 100 / 7 = 14 (et il reste 2)`	quotient prend une nouvelle valeur : a / b. (Le commentaire le dit : « 100 / 7 = 14 (et il reste 2) ».)
`nombre(8, 8, quotient);  // 014`	Appel de la fonction nombre de la console, avec les arguments 8, 8, quotient. Ce qu'elle fait : écrit un nombre calculé, en base dix. Le 4e argument dit combien de chiffres (3 par défaut). Un 5e est la TAILLE, sans limite : nombre(2, 2, score, 3, 4) écrit le score quatre fois plus grand. (Le commentaire le dit : « 014 ».)
`while (true) {`	La boucle du jeu : tout ce qui est entre les accolades recommence sans fin, 60 fois par seconde avec image().
`image();`	Attendre l'image suivante : la console en montre 60 par seconde. C'est ce qui règle la vitesse du jeu.
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **406 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
  démarrage        60 o  ← la console, à l'allumage  — la mise en route de la console
  main()           27 o  ← démarrage  — ta fonction
  VBlank            8 o  ← la console, à chaque image  — compte les images, 60 fois par seconde
  AttendreImage    28 o  ← image()  — attend l'image suivante
  AttendreVBlank   16 o  ← démarrage  — attend que l'écran soit libre
  AttendreAcces    12 o  ← EcrireNombre  — attend de pouvoir écrire une case
  EcrireNombre     33 o  ← nombre()  — écrit un nombre en chiffres
  Diviser          17 o  ← main()  — divise — le processeur ne sait pas le faire
  CopierTuiles     31 o  ← démarrage  — copie les lettres et les dessins en mémoire vidéo
  EffacerCarte     14 o  ← démarrage  — vide les deux cartes de l'écran
  Tuiles          160 o  ← CopierTuiles  — 10 lettres sur 43
Lettres gravées : 0123456789
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireTexte, Reste,
Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau,
LireManette, DonneesTransfert, Notes
Fonctions du langage appelées :
  nombre() × 1    10 o sur place, dans main()  + EcrireNombre (33 o)
  image()  × 1     3 o sur place, dans main()  + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- #include <diviser> : a / b ----

#include <diviser>   // la routine qui divise
#include <nombre>
```

2. UNE VARIABLE — A

```
uint8_t a = 100;
```

3. UNE VARIABLE — B

```
uint8_t b = 7;
```

4. UNE VARIABLE — QUOTIENT

```
uint8_t quotient = 0;
```

5. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

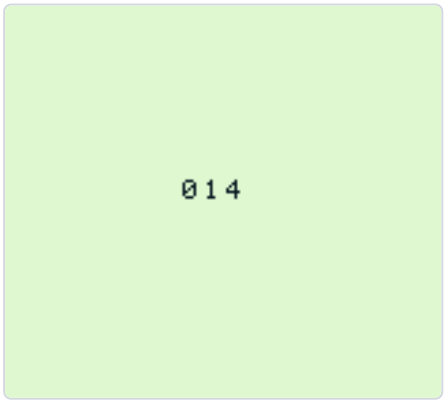
```
int main() {
  quotient = a / b;           // 100 / 7 = 14 (et il reste 2)
  nombre(8, 8, quotient);     // 014

  while (true) {
    image();
  }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.

la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

> 014 au milieu de l'écran.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ 100 / 7 = 14

✓ 014 s'affiche

Ensuite : **0.9. Tout l'alphabet, avec une seule boucle** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++